



PROJECT DOCUMENTATION

E-Learning Course Progress Tracker — OpenBook

Subject: Full Stack Web Development

Project No.: 199

University: ITM Skills University

Semester: Semester III

SUBMITTED BY

Name: Sameer Shaikh

Roll Number: 150096725026

Submission: GitHub Repository

OpenBook: E-Learning Course Progress Tracker

Project summary

OpenBook is a full-stack learning platform built for Case Study 199. Learners browse a course catalog, enroll in a course, read its lessons, mark lessons complete, and review their own progress. Instructors create courses, maintain lesson content, and view aggregate completion information for courses they own. The system records lesson-level progress and derives course completion from that record.

Problem addressed

A course is made up of several lessons, so a single course-level checkbox cannot reliably show what a learner has finished. The application stores completion against individual lessons and calculates a course percentage from the current lesson list. It also has to keep learner records private and reserve course-management operations for the instructor who owns that course.

Project objectives

- Represent users, courses, lessons, and progress as related MongoDB documents using Mongoose.
- Expose a REST API for account access, course and lesson management, enrollment, progress, and instructor analytics.
- Use JWT authentication and role, ownership, and enrollment checks for protected operations.
- Keep completion rules on the server so the browser cannot decide that a course is complete on its own.
- Connect a React/Vite interface to the API and deploy the frontend and backend separately.

Scope and outcome

The delivered scope includes learner and instructor accounts, course discovery, course and lesson CRUD for owners, enrollment, lesson completion, learner progress, instructor completion rates, seed data, and a request collection for API verification. Payments, video hosting, quizzes, password recovery, certificates, and administrator moderation are outside the implemented scope. The project demonstrates the complete request path from browser interaction through Express and Mongoose to MongoDB.

Core rule: a course is complete only when it contains at least one lesson and every current lesson has a completed progress record for that learner.

How success is measured

The principal success measure is consistency between the lesson records and the course summary. If a learner has finished three of four lessons, the API reports 75 percent and keeps the course incomplete. Completing the fourth lesson changes the derived result to 100 percent and complete. An instructor's aggregate uses the same rule for each enrolled learner.

Trust boundaries

The browser is a presentation client and sends user actions to the API. Identity comes from the signed token, while role and ownership are checked against server-side data. The client may display a progress bar, but it cannot grant enrollment or authoritatively change completion without a valid API operation.

Users, permissions, and functional behavior

The application has two roles. A learner consumes course content and owns their individual progress records. An instructor creates and maintains courses and lessons and can view aggregate completion for courses they own. The API—not the visibility of a frontend button—is the final authority for access.

Capability	Learner	Instructor
Browse catalog and course details	Yes	Yes
Enroll in a course	Yes	No
Read lessons	Yes	Yes
Mark lessons complete / see own progress	Enrolled courses	No
Create, edit, or delete owned courses	No	Yes
Manage lessons in an owned course	No	Yes
View aggregate completion analytics	No	Owned courses

Main learner journey

A learner registers or signs in, opens the catalog, selects a course, and enrolls. Enrollment initializes progress entries for that course's lessons. The learner opens the course reader, selects a lesson, and submits completion. The progress page summarizes lesson counts and percentage. A course becomes complete only when every lesson in the course is complete; a zero-lesson course remains incomplete.

Main instructor journey

An instructor signs in, creates a course, adds lessons, and edits or removes course content as it develops. The instructor dashboard lists owned courses and provides access to completion analytics. Ownership is checked against the authenticated user for every course or lesson mutation and analytics request.

Validation and response behavior

Required text fields are checked for missing or blank values; email input is validated and normalized; MongoDB identifiers are validated before database lookups. Responses follow a consistent { message, data } envelope. Typical outcomes include 201 for created resources, 200 for successful reads or updates, 400 for invalid or duplicate actions, 401 for absent or invalid identity, 403 for an authenticated but unauthorized request, and 404 for a missing resource.

Quality goals

The API keeps authorization close to the data operation, returns lightweight catalog records, and uses database constraints to guard against duplicate progress. The frontend presents distinct loading, empty, and error states while retaining the API as the source of truth.

Application structure and request lifecycle

OpenBook uses a conventional client–server architecture. The browser sends JSON over HTTP to the Express API. Routes map each URL and method to authentication, validation, role or resource checks, and a controller. Controllers use Mongoose models to read or change MongoDB documents and return a JSON response.

React + Vite browser → Axios API client → Express routes → Middleware → Controllers → Mongoose models → MongoDB Atlas

Backend module responsibilities

Module	Responsibility
server.js	Loads configuration, configures CORS and JSON parsing, mounts route modules, exposes health and fallback handlers, and listens on the platform-provided port.
config/db.js	Connects Mongoose to the URI in MONGO_URI or MONGODB_URI and reports connection failure.
routes/	Defines auth, course, nested lesson, and progress URL/method mappings.
middleware/	Verifies JWT identity; checks role, ownership, enrollment, required fields, and MongoDB IDs.
controllers/	Implements request validation, business rules, database reads/writes, and response envelopes.
models/	Defines User, Course, Lesson, and Progress schemas, references, defaults, and indexes.

Request lifecycle

1. The user signs in and the API returns a signed JWT with user identity and role.
2. Axios attaches the token as Authorization: Bearer <token> on subsequent requests.
3. Authentication middleware verifies the token and loads the user for req.user.
4. Role, ownership, enrollment, and input middleware run where the route requires them.
5. The controller applies the operation, persists changes through Mongoose, and sends { message, data }.

Response contract and error path

Successful responses place the result in data. Errors are routed through the shared error handler and normally include a concise message with data set to null. Unknown URLs return a route-not-found response. The health endpoint gives deployment monitoring a simple way to confirm that the API process is reachable; database connectivity is also visible in the service logs.

Why controllers remain separate

Route definitions stay focused on URL, method, and middleware order; controllers hold operation behavior. This separation keeps authentication and ownership checks reusable and makes the business rules easier to review. Models describe persisted structure, while the controllers decide when to populate references, create progress records, or remove dependent documents.

MongoDB collections and integrity rules

The database has four application collections. MongoDB ObjectIds are used for references. Course detail and analytics requests populate only the related fields needed for that screen, while course-list responses avoid returning full lesson content.

Collection	Important fields and purpose
User	name, normalized unique email, password hash, role (learner or instructor), timestamps. Password is excluded from ordinary queries by default.
Course	title, description, category and duration metadata, instructor reference, lesson references, enrolled learner references, timestamps.
Lesson	title, content, parent course reference, order, optional duration in minutes, timestamps.
Progress	learner reference, course reference, lesson reference, completed boolean, timestamps. A compound unique index covers learner + lesson.

Relationships

One instructor may own many courses. Each course points to its instructor, references its lessons, and stores the learner IDs enrolled in it. A lesson belongs to one course. A progress document ties a learner and course to one lesson. This design makes the lesson the unit of recorded work while allowing the course to summarize that work.

User (instructor) 1 — * Course 1 — * Lesson
 User (learner) * — * Course through enrollment
 User (learner) 1 — * Progress * — 1 Lesson

Integrity and query choices

Email normalization reduces duplicate accounts that differ only by letter case; the unique database index enforces uniqueness under concurrent requests. Password hashes are created with bcryptjs and are not returned in user responses. The compound progress index prevents duplicate completion rows for a learner and lesson. Enrollment uses an atomic conditional update with \$addToSet so the same learner cannot be added twice by parallel requests. Progress initialization uses bulk upserts, making repeated initialization safe.

Derived values

For T current lessons and C completed lessons, the percentage is $\text{round}(C \div T \times 100)$. Course completion is true only when T is greater than zero and C equals T. Completed counts are calculated against the course's current lesson IDs, so removed lessons do not continue to inflate course progress.

Reference maintenance

Because related records live in separate collections, the application maintains references as documents are created or removed. The lesson controller updates the parent course's lesson array alongside the lesson record, and deletion removes progress tied to that lesson. This explicit cleanup prevents orphaned progress from being counted in later course calculations.

Routes, authentication, and authorization

Registration and login are public. The health endpoint is public for service checks. Course reads and the remaining operations are protected according to their route requirements. The backend uses bearer-token JWT authentication; the frontend stores the returned token with the user session and sends it on API requests.

Method and route	Purpose / access
POST /api/auth/register	Create learner or instructor account; public.
POST /api/auth/login	Validate credentials and return user plus JWT; public.
GET /api/courses	List catalog with instructor and lightweight lesson fields; authenticated.
GET /api/courses/:id	Read one course and its lesson content; authenticated.
POST /api/courses / PUT, DELETE /api/courses/:id	Create as instructor; modify or delete only as course owner.
POST /api/courses/:id/enroll	Enroll current learner and initialize progress.
POST /api/courses/:courseId/lessons	Create lesson for an owned course.
PUT, DELETE /api/courses/:courseId/lessons/:id	Update or remove an owned course lesson.
POST /api/lessons/:id/complete	Mark lesson complete for enrolled learner.
GET /api/courses/:id/progress	Read the enrolled learner's own course progress.
GET /api/courses/:id/completion-rate	Read aggregate completion for the course owner.
GET /api/health	Check API process health; public.

Authentication details

Registration validates required fields, normalizes email, hashes the password with bcryptjs using 10 salt rounds, and issues a JWT. Login retrieves the password hash explicitly, compares it with the submitted password, then signs a token containing the user ID and role. Token lifetime is configured with JWT_EXPIRES_IN and defaults to seven days. The response omits the password hash.

Authorization layers

authMiddleware establishes identity. roleMiddleware restricts routes by learner or instructor role. ownershipMiddleware confirms that the signed-in instructor owns the target course or lesson and that a learner is enrolled before accessing learner progress. validate and validateObjectId guard request bodies and route parameters. Controllers repeat essential domain checks where they need the related course or lesson data to complete the operation safely.

Handling unauthorized requests

A missing or invalid token produces an authentication failure. A valid token with the wrong role, course owner, or enrollment relationship produces a forbidden response. This distinction gives the frontend consistent outcomes and avoids exposing another learner's progress. The frontend interceptor handles 401 by returning the user to login and handles 403 by sending them to the no-access page.

Enrollment, completion, analytics, and cleanup

Enrollment and initial progress

The enrollment endpoint accepts learner requests only. It conditionally updates the course only if the learner is not already enrolled. A missing course returns 404; a duplicate enrollment returns a client error. After enrollment, bulk upserts create incomplete Progress records for the course's lessons. The course response includes the total lesson count so the frontend can display the learner's starting state.

Lesson creation and maintenance

An instructor can create lessons only for a course they own. If order is omitted, the controller finds the highest existing order and assigns the next value. The lesson document is saved and its ID is added to the parent course. Updating a lesson checks that it belongs to the given course and that the instructor owns that course. Deleting a lesson removes its ID from the course and deletes progress records for that lesson.

Completing a lesson

The completion endpoint verifies that the target lesson exists and belongs to a course in which the learner is enrolled. It upserts the learner's Progress record with `completed=true`. Because the operation is an upsert, retrying the request does not create duplicate records. The controller then counts completed progress for the course's current lesson IDs, computes percentage, and returns both progress and `courseCompleted`.

Course progress and empty courses

The progress endpoint returns `totalLessons`, `completedLessons`, `percentage`, `isCourseCompleted`, and lesson-level status. A course with no lessons returns zero totals and zero percent and is not complete. This explicit rule avoids treating "nothing assigned" as successful completion.

Instructor analytics and cleanup

Completion-rate analytics is restricted to the instructor who owns the course. It checks enrolled learners against the same all-lessons-complete rule and returns `totalEnrolled`, `totalCompleted`, and `completionRate`. When a course is deleted, its lessons and progress records are deleted as well. Lesson deletion similarly removes related progress. The current cleanup uses multiple database writes rather than a MongoDB transaction; that is a practical area to harden if the service grows.

Progress records are the source of truth. Course percentage and completion status are derived from the learner's completed lessons, not maintained as an independently editable flag.

Repeat requests and changing curricula

Marking a lesson complete more than once is safe because the unique learner-and-lesson key and upsert behavior converge on one record. When an instructor removes a lesson, its progress record is deleted and the course's completion calculation considers only the remaining lesson references. Adding a lesson increases the total, so learners must complete the new lesson before the course can be complete again.

How the browser connects to the service

Frontend responsibilities

The frontend is a React application built with Vite and JavaScript. React Router separates public pages from authenticated learner and instructor views. The shared shell contains role-appropriate navigation. The catalog supports course discovery; course detail combines curriculum selection and lesson reading; learner pages show enrolled-course progress; instructor pages support course and lesson management and completion analytics.

AuthContext persists the authenticated user and token in localStorage. The Axios API layer uses VITE_API_BASE_URL, adds the bearer token to requests, and redirects for authentication and access errors. Page components request data from the API and present loading skeletons, empty results, errors with retry, and successful content. The frontend does not calculate authoritative completion or bypass API permissions.

Local configuration

Service	Configuration / command
Backend	PORT, MONGO_URI (or MONGODB_URI), JWT_SECRET, JWT_EXPIRES_IN are supplied through environment variables. Run with npm start; use npm run dev during development.
Database	MongoDB Atlas connection URI is stored in the backend runtime environment. npm run seed runs the course and lesson seed script.
Frontend	VITE_API_BASE_URL sets the API base URL. npm run dev starts Vite on port 5173; npm run build creates the production bundle.
Local API proxy	The Vite development configuration proxies /api to the local Express service on port 5001.

Deployment arrangement

The backend is deployed as a Render web service, MongoDB is hosted by MongoDB Atlas, and the React frontend is deployed on Vercel. The API reads Render's assigned PORT and connects to Atlas using the configured environment URI. The frontend build receives the deployed API URL through VITE_API_BASE_URL. A Vercel rewrite serves the React entry point for direct navigation to client-side routes.

Operational verification

The deployed health route is <https://e-learning-course-progress-tracker.onrender.com/api/health>. During deployment, the Render logs showed the service listening on its assigned port and MongoDB connecting successfully. The health endpoint returned status OK. The live frontend was confirmed to retrieve catalog data through the deployed API.

Testing record and conclusion

API verification

The Postman collection is organized into numbered folders covering system checks, authentication, courses, lessons, enrollment and progress, zero-lesson courses, cleanup, and additional checks. The Newman run completed 60 of 60 requests successfully. The cases exercise both successful and rejected operations, including role restrictions, ownership, invalid identifiers, duplicate enrollment, completion calculation, and cascading cleanup.

End-to-end checks

Manual browser checks confirmed that the deployed frontend opens, requests courses from the Render API, and supports learner enrollment and lesson-progress interactions. Instructor-facing course management and completion analytics were also checked. The API health route and Render logs confirmed the service and database connection after deployment.

Scenarios covered

Area	Behavior checked
Authentication	Valid registration and login, invalid credentials, and protected requests without a valid token.
Courses and lessons	Create, read, update, and delete operations; required fields and invalid identifiers.
Roles and ownership	Learner and instructor role restrictions, course ownership, lesson-course matching, and enrollment checks.
Enrollment and progress	Successful and duplicate enrollment, lesson completion, personal progress, and completion percentage.
Boundary and cleanup cases	Zero-lesson courses, course completion only after every lesson, and removal of dependent progress records.

Known limitations

- Registration accepts either the learner or instructor role; instructor registration does not require a separate approval step.
- Authenticated users can read course and lesson details without enrollment; enrollment is required for progress operations.
- The API does not include rate limiting or password recovery.
- Enrollment progress initialization and course or lesson cascade deletion use multiple writes rather than a MongoDB transaction.
- Instructor completion analytics checks enrolled learners individually.

Conclusion

OpenBook implements the central Case Study 199 outcome: learners can enroll and record lesson completion, while course completion is determined only after every current lesson is complete. JWT authentication, role checks, ownership checks, and enrollment checks protect the API operations. The project combines a modular Express backend, referenced Mongoose models, a React client, a Newman request collection, and a deployed service stack.

GitHub Repository Link

<https://github.com/sameershaikh-20/e-learning-course-progress-tracker>